GLO Niveau 5 Alternance

Architecture Logicielle & Objets répartis – 2026-2027

Enseignant : M. Tchuigoua Elvé

Groupe1 : zendesck api; gestion des courriers dans une agence

NOM	MATRICULE
DITCHO SITCHEU ELFRIED FIONA	24G01671
DONGMO NGOUAJIO HAROLD BOTREL	24GO1752
AICHA MOHAMMED KABIR BAKO	24G01669
DJOUKA FONGANG FREDERIC MELISSA	24G01672
DOMENI NGAMENI WILLIAM STEVE	24G01750
DIPANDA LUC MIGUEL	24G01670
EBOA EPEE STYVE AURELIEN	24G01754
EKOKO BINDZI EMMANUEL THURAM	24G01856

github projet : https://github.com/ElfriedFiona/GestionDesCouriers.git

CAHIER DES CHARGES : GESTION ÉLECTRONIQUE DES COURRIERS ET RÉCLAMATIONS (GEC-AGENCE)

1. Présentation du Projet et Contexte Métier

1.1 Contexte et Enjeux

La transformation numérique des entreprises, grandes agences et administrations publiques, ainsi que l'évolution continue des attentes des usagers et clients, imposent une modernisation profonde de la relation usager/client. Les canaux de communication se sont multipliés, générant un flux massif et hétérogène de demandes et de courriers entrants chaque jour par diverses voies : e-mails officiels adressés à l'agence, courriers physiques scannés et numérisés en guichet ou bureau d'accueil, formulaires soumis depuis le portail web ou l'application mobile, ainsi que réclamations directement enregistrées aux points de contact. Face à cette dispersion, la centralisation omnicanale constitue un enjeu majeur pour éliminer les risques de perte de documents, supprimer les doublons de traitement et offrir une vue à 360 degrés de l'historique des échanges avec chaque usager ou client.

Par ailleurs, toute grande agence évolue dans un cadre réglementaire et légal strict (délais de réponse légaux, charte de qualité, normes RGPD, obligations de transparence). Les autorités de tutelle et organes de contrôle imposent des normes rigoureuses quant au traitement des requêtes et réclamations, assorties de délais plafonds de réponse obligatoires et de sanctions pécuniary ou administratives en cas de manquement. Le respect scrupuleux de ces exigences réglementaires exige la mise en place d'un outil de Gestion Électronique des Courriers (GEC) hautement performant, capable d'horodater l'ensemble des réceptions, d'automatiser le suivi des échéances et de fournir aux auditeurs internes et externes des pistes d'audit auditables et inaltérables.

1.2 Objectifs Stratégiques

Le projet GEC-AGENCE vise à répondre aux objectifs stratégiques suivants :

- **Réduction significative du temps moyen de traitement :** Diminuer drastiquement les délais d'instruction et de résolution des réclamations en automatisant les tâches manuelles de saisie et de transmission inter-services.
- **Garantie de la traçabilité intégrale et inviolabilité de l'historique :** Assurer la traçabilité de chaque action réalisée sur un dossier (consultation, annotation, transmission, validation) afin de garantir la valeur probante du dossier client.
- **Automatisation du routage intelligent :** Qualifier dès l'enrôlement la nature du document et l'orienter automatiquement vers les entités métiers compétentes (Direction Juridique et Contentieux, Service Relation Client/Usager, Service Administratif, Unité de Conformité).
- **Sécurisation maximale des données sensibles :** Appliquer un niveau élevé de sécurité conformément aux normes institutionnelles et réglementations de protection des données personnelles (chiffrement, gestion stricte des habilitations et respect de la confidentialité des dossiers).

1.3 Périmètre d'Application

Le périmètre fonctionnel du système couvre la totalité des flux documentaires et des requêtes entrant ou sortant des structures régionales, sites opérationnels et du siège de l'agence :

- **Courriers entrants et sortants :** Prise en charge des courriers postaux numérisés, e-mails, communication web, bordereaux d'envoi et lettres d'information adressées aux usagers, clients ou tiers.
- **Traitement des réclamations et requêtes :** Centralisation et instruction des demandes relatives aux litiges de facturation, contestations de services, dysfonctionnements de prestations et erreurs administratives.
- **Suivi des dossiers administratifs et contractuels :** Instruction des requêtes liées aux contrats, pièces justificatives, démarches réglementaires et demandes d'autorisations.
- **Circulation des notes d'information internes :** Numérisation et suivi de la diffusion des directives, circulaires et notes de service inter-agences ou inter-départements.

2. Processus Métier et Typologie des Flux

2.1 Typologie des Flux

Afin d'assurer un traitement rapide et précis, la plateforme classe les flux d'agences en quatre grandes catégories :

- **Réclamations usagers / clients :** Comprend les litiges relatifs aux prestations, contestations de factures ou de frais, ainsi que les signalements d'anomalies de service.
- **Demandes de services ou de documents :** Regroupe les demandes d'attestations, la fourniture de documents officiels, et les mises à jour réglementaires des dossiers d'usagers ou partenaires.
- **Correspondances réglementaires et institutionnelles :** Traitement prioritaire des requêtes émanant des autorités de contrôle, rapports d'audit, ainsi que de l'exercice du droit de communication par les autorités judiciaires et administratives.
- **Courriers internes :** Transmission et archivage des notes d'orientation stratégique de la Direction Générale, des circulaires internes et des instructions opérationnelles entre les entités et départements.

2.2 Modélisation des Workflows (BPMN)

Les circuits opérationnels de prise en charge des flux respectent des règles métier préétablies :

- **Circuits de traitement différenciés :** Les réclamations de niveau standard sont orientées vers les gestionnaires N1/N2 dans les services concernés. Les dossiers qualifiés de sensibles ou urgents (contentieux importants, risque d'atteinte à la réputation de l'agence) basculent immédiatement dans un circuit prioritaire avec notification instantanée des responsables d'unité.
- **Escalade hiérarchique automatique :** En cas d'approches des échéances limites réglementaires (SLA), le système déclenche des alertes progressives destinées au gestionnaire, puis à son supérieur hiérarchique direct si aucun traitement n'est enregistré dans un délai prescrit.

- **Validation multi-niveaux et signature électronique :** Toute proposition de réponse officielle à un client ou à un tiers doit suivre un circuit de validation ajustable selon le montant des enjeux. Une fois validée, la réponse est signée électroniquement avant son expédition.

2.3 Matrice des Rôles et Responsabilités (RACI)

Le tableau ci-dessous synthétise la répartition des responsabilités au sein du système GEC-AGENCE :

Rôle / Acteur	Réception & Qualification	Instruction & Rédaction	Validation & Avis	Escalade & Arbitrage
Agent d'accueil / Chargé de relation	R (Réalisateur)	C (Consulté)	I (Informé)	I (Informé)
Gestionnaire Back-Office	C (Consulté)	R (Réalisateur)	I (Informé)	I (Informé)
Chef de Service / Responsable Unité	I (Informé)	A (Approbateur)	A (Approbateur)	C (Consulté)
Responsable Conformité / Juridique	I (Informé)	C (Consulté)	A (Approbateur)	C (Consulté)
Directeur d'entité / Région	I (Informé)	I (Informé)	C (Consulté)	A (Approbateur)

3. Spécifications Fonctionnelles

3.1 Module d'Enrôlement et d'Indexation (Guichet et Back-Office)

Le module de saisie et d'indexation constitue la porte d'entrée principale des dossiers :

- **Liaison automatique à la fiche usager / client :** Saisie unifiée permettant la recherche rapide et le rattachement direct de la réclamation ou du courrier à la fiche usager/client unique via le numéro d'identifiant, le code dossier ou le nom.

- **Numérisation, OCR et téléversement :** Intégration d'un moteur OCR (Reconnaissance Optique de Caractères) pour numériser, indexer automatiquement le texte et téléverser les pièces justificatives (pièces d'identité, formulaires, contrats, formulaires administratifs) sous formats sécurisés (PDF/A, PNG).

3.2 Intégration et Moteur de Ticketing

Le moteur de ticketing centralise la gestion des flux sous forme de tickets informatisés :

- **Génération automatique de ticket :** Création automatique d'un ticket unique dès l'enregistrement d'un courrier avec association de toutes les métadonnées d'agences (structure de rattachement, segment, type de produit/service, niveau de priorité).
- **Gestion dynamique des statuts :** Maintien d'un cycle de vie clair synchronisé entre l'application métier et le moteur de ticketing avec des états prédéfinis : *Nouveau* \rightarrow *En instruction* \rightarrow *En attente de validation* \rightarrow *Résolu* \rightarrow *Clôturé*.

3.3 Gestion des SLA et des Alertes Réglementaires

Afin d'assurer le respect des exigences des autorités de tutelle, le système gère les éléments suivants :

- **Configuration des règles SLA :** Application de délais stricts de prise en charge et de résolution selon la typologie (ex. 48h ouvrées pour les requêtes urgentes, 5 jours ouvrés pour les réclamations standard).
- **Tableaux de bord et KPI :** Mises à disposition de tableaux de bord en temps réel affichant les indicateurs clés de performance (taux de résolution au premier contact, durée moyenne d'instruction, taux de respect des SLA par entité et par région).

3.4 Sécurité et Confidentialité (RBAC)

Le modèle d'habilitation repose sur un contrôle d'accès basé sur les rôles (Role-Based Access Control - RBAC). Il garantit le respect absolu de la confidentialité et le cloisonnement strict des données personnelles sensibles. Seuls les agents dûment habilités et affectés au périmètre d'un dossier peuvent en consulter le contenu ou les pièces jointes.

4. Spécifications Techniques et Architecture

4.1 Architecture Distribuée (Modèle Façade / Middleware)

La solution s'appuie sur l'API REST de Zendesk (api/v2) intégrée via une architecture distribuée à microservices. Le Middleware/Façade API sécurisé relaie l'ensemble des requêtes entre l'application front-end de l'agence et les endpoints Zendesk :

- **Endpoints REST Tickets :** Utilisation de `POST /api/v2/tickets.json` pour la création, `PUT /api/v2/tickets/{id}.json` pour la mise à jour des statuts/champs, et `POST /api/v2/tickets/create_many.json` pour les traitements par lots.
- **Endpoints Utilisateurs et Agences :** Exploitation de `GET /api/v2/users/search.json` et `POST /api/v2/users.json` pour la synchronisation des fiches usagers/clients et agents.
- **Champs Personnalisés (Custom Fields) Métiers :** Structuration fine des métadonnées métier dans les tickets Zendesk via des champs personnalisés dédiés : Identifiant Usager/Client, Numéro de Dossier, Type de Service, Niveau de Priorité et Impact.
- **Webhooks Temps Réel :** Configuration de déclencheurs (triggers) et webhooks HTTP vers le Middleware pour notifier en temps réel l'application métier lors de chaque changement d'état ou d'escalade d'un ticket Zendesk.
- **Authentification et Sécurité API :** Connexions sécurisées à l'API Zendesk via un jeton API (API Token avec authentification Basic) ou OAuth 2.0 avec portée restreinte aux ressources nécessaires.

Résilience et Gestion du Rate Limiting : Traitement des limites de requêtes (rate limits HTTP 429 de Zendesk) au niveau du Middleware par mise en œuvre d'un mécanisme de rejeu avec délai d'attente exponentiel (exponential backoff), de files d'attente asynchrones et d'un pattern Circuit Breaker pour garantir la haute disponibilité.

4.2 Sécurité et Conformité des Données

Les exigences de sécurité intègrent les standards de la profession et les règles de protection des données :

- **Chiffrement fort :** Utilisation du protocole TLS 1.3 pour l'ensemble des communications réseau en transit et algorithme AES-256 pour le chiffrement des données et pièces jointes au repos.
- **Piste d'audit immuable :** Journalisation inaltérable et horodatée de toute opération de création, consultation, modification ou suppression relative à un dossier client afin de faciliter les contrôles d'audit.

4.3 Résilience et Continuité d'Activité

Pour garantir une disponibilité de 99,9%, l'architecture est déployée en haute disponibilité sur deux centres de données distants avec basculement automatique (failover). Le traitement des événements s'effectue via des files d'attente asynchrones afin d'éviter toute perte de données lors des pics d'activité ou d'interruptions réseau temporaires.

5. Planification et Démarche de Réalisation

5.1 Méthodologie Agile

Le projet sera conduit selon la méthodologie Agile/Scrum organisée en Sprints de 2 à 3 semaines. À la fin de chaque sprint, des démonstrations des fonctionnalités développées seront réalisées auprès des représentants des départements métiers (Exploitation, Conformité, Service Client).

5.2 Jalons de Déploiement

Le projet se déroulera selon quatre phases principales :

- **Phase 1 - Cadrage et Conception :** Validation des spécifications détaillées, du modèle de données et des architectures de sécurité.

- **Phase 2 - Développement et Intégration :** Implémentation des connecteurs d'API, configuration du moteur de ticketing et du portail guichet.
- **Phase 3 - Déploiement Pilote :** Lancement opérationnel sur un panel restreint d'agences représentatives pour ajustement des workflows.
- **Phase 4 - Généralisation :** Déploiement progressif à l'ensemble du réseau d'agences et des départements du siège.

5.3 Stratégie de Tests

La stratégie globale de recette et validation technique repose sur :

- **Tests unitaires et d'intégration :** Vérification automatisée de l'ensemble des composants du code et de la transmission correcte des payloads d'API.
- **Tests de sécurité et d'intrusion (Pentests) :** Audits de code source et tests d'intrusion menés par des experts indépendants pour valider la résistance de la solution aux cyberattaques.
- **Tests de charge :** Simulation de volumes massifs d'enrôlement et de consultations simultanées pour vérifier la stabilité du système sous forte contrainte.

6. Livrables et Recette

6.1 Livrables Logiciels

Les livrables applicatifs comprennent le code source complet de l'application web métier, les modules connecteurs d'API sécurisés, ainsi que l'ensemble des scripts automatisés de configuration des règles métiers, des déclencheurs et des alertes de notification.

6.2 Documentation

Une documentation complète sera remise à la livraison : guides de déploiement et d'administration système, manuels d'utilisation destinés aux agents et responsables, Dossier d'Architecture Technique (DAT) et cartographie exhaustive des flux de données.

6.3 Cahier de Recette

Le cahier de recette fonctionnelle définira les scénarios détaillés d'homologation permettant de valider le respect de la confidentialité des données, la conformité des calculs de SLA, la précision du routage automatique et la qualité irréprochable des pistes d'audit.

DOCUMENT D'ANALYSE COMPLET: PROJET GEC-AGENCE

1. Présentation du Projet et Contexte Métier

1.1 Contexte et Enjeux

Le présent document constitue l'analyse complète et structurée du projet de Gestion Électronique des Courriers et Réclamations (GEC-AGENCE). Face à la transformation numérique et à la multiplication des canaux de communication (e-mails officiels, courriers physiques numérisés, portail web, application mobile, guichets d'accueil), l'agence doit centraliser et sécuriser la prise en charge de ses flux documentaires. L'objectif global de cette analyse est d'évaluer le cadre opérationnel, fonctionnel, technique et de sécurité permettant l'intégration transparente de la plateforme Helpdesk Zendesk (via son API REST) au cœur du système d'information d'agence.

La démarche de cadrage s'inscrit dans un strict respect des exigences réglementaires et des normes du secteur. Le projet vise à éliminer tout risque de perte de documents, à garantir des délais plafonds de réponse (SLA), à automatiser le routage vers les directions métiers compétentes (Finances, Juridique, Conformité, Service client) et à assurer une traçabilité totale et inaltérable à des fins d'audit interne et externe.

1.2 Objectifs Stratégiques

Le projet GEC-AGENCE vise à répondre aux objectifs stratégiques suivants :

- **Réduction significative du temps moyen de traitement :** Diminuer drastiquement les délais d'instruction et de résolution des demandes en automatisant les tâches manuelles de saisie et de transmission inter-services.
- **Garantie de la traçabilité intégrale et inviolabilité de l'historique :** Assurer la traçabilité de chaque action réalisée sur un dossier (consultation, annotation, transmission, validation) afin de garantir la valeur probante du dossier client.

- **Automatisation du routage intelligent :** Qualifier dès l'enrôlement la nature du document et l'orienter automatiquement vers les entités métiers compétentes (Service Approvisionnements, Service Juridique, Service Clientèle, Unité de Conformité).
- **Sécurisation maximale des données sensibles :** Appliquer un niveau élevé de sécurité conformément aux normes industrielles et réglementations de protection des données personnelles (chiffrement, gestion stricte des habilitations et respect de la confidentialité).

1.3 Périmètre d'Application

Le périmètre fonctionnel du système couvre la totalité des flux documentaires et des requêtes entrant ou sortant du réseau d'agences, des directions régionales et du siège de l'agence :

- **Courriers entrants et sortants :** Prise en charge des courriers postaux numérisés, e-mails, bordereaux d'envoi et lettres d'information adressées aux clients ou tiers.
- **Traitement des réclamations et requêtes :** Centralisation et instruction des demandes relatives aux incidents de service, erreurs de facturation et contestations diverses.
- **Suivi des dossiers et contrats :** Instruction des requêtes liées aux demandes de renouvellement, contrats, avenants et pièces administratives.
- **Circulation des notes d'information internes :** Numérisation et suivi de la diffusion des directives, circulaires et notes de service inter-agences ou inter-départements.

2. Processus Métier et Typologie des Flux

2.1 Typologie des Flux

L'analyse métiers met en évidence quatre grandes typologies de flux documentaires à traiter de manière omnicanale :

- **Réclamations clients :** Comprend les litiges relatifs aux services délivrés, les contestations de facturation, ainsi que les erreurs d'imputation de frais.
- **Demandes de services ou de documents :** Regroupe les demandes d'attestations de solde ou de capacité financière, la fourniture de relevés de compte historiques

complexes, et les mises à jour réglementaires des dossiers clients (KYC - Know Your Customer).

- **Correspondances réglementaires et institutionnelles :** Traitement prioritaire des requêtes émanant des autorités de contrôle, des rapports d'auditeurs externes, ainsi que des demandes légales des autorités judiciaires et administratives.
- **Courriers internes :** Transmission et archivage des notes d'orientation stratégique de la Direction Générale, des circulaires relatives à la conformité et des instructions opérationnelles entre le siège et les agences du réseau.

2.2 Modélisation des Workflows (BPMN)

Les workflows opérationnels sont modélisés selon trois axes fonctionnels fondamentaux :

- **Circuits de traitement différenciés :** Les réclamations de niveau standard sont orientées vers les gestionnaires N1/N2 en agence ou back-office. Les dossiers qualifiés de sensibles ou urgents (incidents majeurs, préjudices financiers élevés, risque d'atteinte à la réputation) basculent immédiatement dans un circuit prioritaire avec notification instantanée des responsables d'unité.
- **Escalade hiérarchique automatique :** En cas d'approches des échéances limites réglementaires (SLA), le système déclenche des alertes progressives destinées au gestionnaire, puis à son supérieur hiérarchique direct si aucun traitement n'est enregistré dans un délai prescrit.
- **Validation multi-niveaux et signature électronique :** Toute proposition de réponse officielle à un client ou à un tiers doit suivre un circuit de validation ajustable selon le montant des enjeux. Une fois validée, la réponse est signée électroniquement avant son expédition.

2.3 Matrice des Rôles et Responsabilités (RACI)

Le tableau ci-dessous synthétise la répartition des responsabilités au sein du système GEC-AGENCE :

Rôle / Acteur	Réception & Qualification	Instruction & Rédaction	Validation & Avis	Escalade & Arbitrage
Chargé de clientèle en agence	R (Réalisateur)	C (Consulté)	I (Informé)	I (Informé)
Gestionnaire Back-Office	C (Consulté)	R (Réalisateur)	I (Informé)	I (Informé)
Chef de Service / Responsable Unité	I (Informé)	A (Approbateur)	A (Approbateur)	C (Consulté)
Officier de Conformité	I (Informé)	C (Consulté)	A (Approbateur)	C (Consulté)
Directeur d'agence / Réseau	I (Informé)	I (Informé)	C (Consulté)	A (Approbateur)

3. Spécifications Fonctionnelles

3.1 Module d'Enrôlement et d'Indexation (Guichet et Back-Office)

Le module de saisie et d'indexation constitue la porte d'entrée principale des dossiers :

- **Liaison automatique à la fiche client :** Saisie unifiée permettant la recherche rapide et le rattachement direct de la réclamation ou du courrier à la fiche client unique via le numéro d'identification, le code client ou le nom.
- **Numérisation, OCR et téléversement :** Intégration d'un moteur OCR (Reconnaissance Optique de Caractères) pour numériser, indexer automatiquement le texte et téléverser les pièces justificatives (pièces d'identité, contrats, justificatifs de domicile, factures) sous formats sécurisés (PDF/A, PNG).

3.2 Intégration et Moteur de Ticketing

Le moteur de ticketing centralise la gestion des flux sous forme de tickets informatisés :

- **Génération automatique de ticket :** Création automatique d'un ticket unique dès l'enregistrement d'un courrier avec association de toutes les métadonnées de l'agence (agence de rattachement, segment client, type de produit/service, niveau de risque).

- **Gestion dynamique des statuts :** Maintien d'un cycle de vie clair synchronisé entre l'application métier et le moteur de ticketing avec des états prédéfinis : *Nouveau* \rightarrow *En instruction* \rightarrow *En attente de validation* \rightarrow *Résolu* \rightarrow *Clôturé*.

3.3 Gestion des SLA et des Alertes Réglementaires

Afin d'assurer le respect des exigences des autorités de tutelle, le système gère les éléments suivants :

- **Configuration des règles SLA :** Application de délais stricts de prise en charge et de résolution selon la typologie (ex. 24h ouvrées pour les réclamations urgentes, 5 jours ouvrés pour les demandes standard).
- **Tableaux de bord et KPI :** Mises à disposition de tableaux de bord en temps réel affichant les indicateurs clés de performance (taux de résolution au premier contact, durée moyenne d'instruction, taux de respect des SLA par agence et par région).

3.4 Sécurité et Confidentialité (RBAC)

L'analyse de sécurité impose des exigences strictes en matière de protection des données sensibles et de confidentialité :

- **Contrôle d'accès basé sur les rôles (RBAC) :** Habilitations fines selon le principe du moindre privilège, cloisonnant l'accès aux données des clients uniquement aux agents et responsables affectés au dossier.
- **Cryptographie et chiffrement fort :** Protection systématique des flux en transit via TLS 1.3 et chiffrement AES-256 pour les données au repos et pièces jointes stockées.
- **Piste d'audit et traçabilité inaltérable :** Horodatage et journalisation immuable de chaque opération (lecture, création, modification, transmission) pour répondre aux audits de conformité.

4.4 Analyse des Risques et Stratégie de Mitigation

Une matrice d'évaluation des risques identifie les menaces majeures et leurs mesures de mitigation :

Risque Identifié	Impact	Probabilité	Stratégie de Mitigation
Dépassement du Rate Limit API Zendesk	Élevé	Moyenne	Mise en place de files d'attente asynchrones (Message Queue), politique de retry avec exponential backoff et mise en cache.
Rupture de la confidentialité / Fuite de données	Critique	Faible	Contrôle d'accès RBAC strict, anonymisation/pseudonymisation des données clients avant transmission, chiffrement TLS 1.3 / AES-256.
Non-respect des délais SLA réglementaires	Élevé	Moyenne	Escalade hiérarchique automatique, notifications push/email en temps réel et tableaux de bord KPI de suivi d'échéances.
Indisponibilité du service Middleware / Cloud	Élevé	Faible	Déploiement multi-datacenter en haute disponibilité (99.9%) avec basculement automatique (failover) et Circuit Breaker.

4. Spécifications Techniques et Architecture

4.1 Architecture Distribuée (Modèle Façade / Middleware)

L'architecture technique s'appuie sur le pattern Middleware/Façade API distribué pour isoler le Système d'Information Central des endpoints REST de Zendesk (api/v2). Le middleware gère la transformation des payloads, la sécurité des accès et la résilience :

- **Endpoints REST Tickets, Agences et Utilisateurs :** Appel des routes `POST /api/v2/tickets.json` pour la création, `PUT /api/v2/tickets/{id}.json` pour le cycle de vie, `GET /api/v2/organizations.json` pour la gestion des agences, et `GET /api/v2/users/search.json` pour la synchronisation des fiches clients.
- **Champs Personnalisés (Custom Fields) :** Structuration des métadonnées métiers (ID Client, Code Agence, Niveau de priorité, Montant de la réclamation) au sein des objets tickets Zendesk.

- **Webhooks et Événements Temps Réel :** Utilisation de déclencheurs (triggers) et webhooks HTTP asynchrones émis par Zendesk vers le middleware pour répercuter instantanément les changements d'état vers les applications bancaires.
- **Webhooks Temps Réel :** Configuration de déclencheurs (triggers) et webhooks HTTP vers le Middleware pour notifier en temps réel l'application métier lors de chaque changement d'état ou d'escalade d'un ticket Zendesk.
- **Authentification et Sécurité API :** Connexions sécurisées à l'API Zendesk via un jeton API (API Token avec authentification Basic) ou OAuth 2.0 avec portée restreinte aux ressources nécessaires.

Gestion du Rate Limiting et Résilience : Pour pallier les contraintes de limites de requêtes (HTTP 429), le middleware intègre un mécanisme de rejeu à délai d'attente exponentiel (exponential backoff), des files d'attente asynchrones message-driven et un disjoncteur (Circuit Breaker Pattern) garantissant la tolérance aux pannes et la haute disponibilité.

4.2 Sécurité et Conformité des Données

Les exigences de sécurité intègrent les standards de la profession :

- **Chiffrement fort :** Utilisation du protocole TLS 1.3 pour l'ensemble des communications réseau en transit et algorithme AES-256 pour le chiffrement des données et pièces jointes au repos.
- **Piste d'audit immuable :** Journalisation inaltérable et horodatée de toute opération de création, consultation, modification ou suppression relative à un dossier client afin de faciliter les contrôles d'audit.

4.3 Résilience et Continuité d'Activité

Pour garantir une disponibilité de 99,9%, l'architecture est déployée en haute disponibilité sur deux centres de données distants avec basculement automatique (failover). Le traitement des événements s'effectue via des files d'attente asynchrones afin d'éviter toute perte de données lors des pics d'activité ou d'interruptions réseau temporaires.

5. Planification et Démarche de Réalisation

5.1 Méthodologie Agile

Le projet sera conduit selon la méthodologie Agile/Scrum organisée en Sprints de 2 à 3 semaines. À la fin de chaque sprint, des démonstrations des fonctionnalités développées seront réalisées auprès des représentants des départements métiers (Exploitation, Conformité, Service Client).

5.2 Jalons de Déploiement

Le projet se déroulera selon quatre phases principales :

- **Phase 1 - Cadrage et Conception :** Validation des spécifications détaillées, du modèle de données et des architectures de sécurité.
- **Phase 2 - Développement et Intégration :** Implémentation des connecteurs d'API, configuration du moteur de ticketing et du portail guichet.
- **Phase 3 - Déploiement Pilote :** Lancement opérationnel sur un panel restreint d'agences représentatives pour ajustement des workflows.
- **Phase 4 - Généralisation :** Déploiement progressif à l'ensemble du réseau d'agences et des départements du siège.

5.3 Stratégie de Tests

La stratégie globale de recette et validation technique repose sur :

- **Tests unitaires et d'intégration :** Vérification automatisée de l'ensemble des composants du code et de la transmission correcte des payloads d'API.
- **Tests de sécurité et d'intrusion (Pentests) :** Audits de code source et tests d'intrusion menés par des experts indépendants pour valider la résistance de la solution aux cyberattaques.
- **Tests de charge :** Simulation de volumes massifs d'enrôlement et de consultations simultanées pour vérifier la stabilité du système sous forte contrainte.

6. Livrables et Recette

6.1 Livrables Logiciels

Les livrables applicatifs comprennent le code source complet de l'application web métier, les modules connecteurs d'API sécurisés, ainsi que l'ensemble des scripts automatisés de configuration des règles métiers, des déclencheurs et des alertes de notification.

6.2 Documentation

Une documentation complète sera remise à la livraison : guides de déploiement et d'administration système, manuels d'utilisation destinés aux agents et responsables, Dossier d'Architecture Technique (DAT) et cartographie exhaustive des flux de données.

6.3 Cahier de Recette

Le cahier de recette fonctionnelle définira les scénarios détaillés d'homologation permettant de valider la confidentialité des échanges, la conformité des calculs de SLA, la précision du routage automatique et la qualité irréprochable des pistes d'audit.

Dossier de conception technique : API de gestion des courriers (ZANDESK API)

1. Introduction

1.1 Objet du document

Ce dossier décrit **comment** l'API de gestion des courriers est conçue et réalisée. Il complète le cahier des charges (*quoi* et *pourquoi*) et le document d'analyse (fonctionnement métier détaillé) : il précise l'architecture, les données, les interfaces, les workflows, la sécurité et la résilience de la solution.

1.2 Périmètre

La solution est une **API Courriers** qui joue le rôle de façade devant le moteur de ticketing Zendesk. Elle offre des fonctions adaptées au métier (enregistrer un courrier, lui attribuer une référence, consulter le registre) et s'appuie, en arrière-plan, sur l'**API REST Zendesk** pour créer et suivre les tickets.

API	Rôle
API Courriers	Conçue par le projet, exposée aux applications clientes
API Zendesk	Service existant, appelé par l'API Courriers

- **Types de courriers** : arrivants (lettres numérisées, e-mails officiels, formulaires web), départs (réponses ou lettres d'initiative nécessitant validation), internes (notes de service, bordereaux de transmission).
- **Hors périmètre** : les écrans. Les applications clientes (portail d'agence, par exemple) consomment l'API ; elle se teste avec Swagger UI ou Postman.

1.3 Acteurs

Acteur	Rôle
Agent du Bureau d'Ordre / Secrétariat	Enregistre, numérise et qualifie les courriers reçus
Agent traitant (agent Zendesk)	Traite le dossier, rédige le projet de réponse
Validateur / Responsable d'unité	Valide les projets de réponse ou réaffecte les dossiers
Administrateur	Gère les paramétrages, les accès et les workflows

2. Architecture globale et composants

2.1 Vue d'ensemble

La solution repose sur une architecture distribuée à trois tiers basée sur un modèle Façade/Middleware pour séparer l'application métier bancaire du moteur de ticketing cloud Zendesk :

- **Front-End Métier (Portail Agence & Back-Office) :** Application web Angular/React sécurisée utilisée par les chargés de clientèle et gestionnaires pour numériser, saisir, instruire et valider les réclamations.
- **Middleware / Façade API Bancaire :** Microservices développés en Java (Spring Boot) / Node.js. Ce composant orchestre les flux métier, applique les règles de sécurité du secret bancaire, gère la persistance locale d'audit et relaie de façon sécurisée les requêtes vers l'API Zendesk.
- **Moteur de File d'Attente & Message Broker (RabbitMQ / Apache Kafka) :** Garantit le traitement asynchrone des flux documentaires et offre une haute résilience en cas de rupture de connectivité avec l'API Zendesk.
- **Moteur de Ticketing Cloud (Zendesk API v2) :** Fournit la gestion centralisée du cycle de vie des tickets, le calcul automatisé des SLA, et l'historique d'interaction.
- **Base de Données Locale (PostgreSQL & Redis) :** Stocke le journal immuable des pistes d'audit, la table de correspondance des identifiants bancaires/Zendesk, ainsi que les caches de performance.

2.2 Composants

Composant	Rôle	Technologie
API Courriers (façade)	Interface REST, règles métier, génération des références, orchestration des appels vers Zendesk	Java (Spring Boot)
File de messages	Traitement asynchrone des appels vers Zendesk, sans perte si Zendesk est indisponible	RabbitMQ
Base locale	Courriers, pièces jointes, correspondance d'identifiants, journal d'audit immuable	PostgreSQL
Cache	Accélération des lectures, compteurs de quotas	Redis
Stockage des fichiers	Pièces jointes numérisées, chiffrées au repos	Système de fichiers ou stockage objet
Moteur de ticketing	Cycle de vie des tickets, SLA, déclencheurs, historique d'interaction	Zendesk, API REST v2

2.3 Flux principaux

1. **Enregistrement d'un courrier** : le client appelle l'API ; elle enregistre le courrier dans la base, publie un message dans la file, puis un traitement appelle Zendesk pour créer le ticket et conserve son numéro.
2. **Retour de Zendesk** : quand un ticket change, Zendesk notifie l'API par webhook ; l'API vérifie la requête, retrouve le courrier et met à jour son statut.
3. **Consultation du registre** : elle est servie par la base locale, sans appel à Zendesk, ce qui la rend rapide et indépendante de la disponibilité de Zendesk.

2.4 Choix de conception

- **Façade plutôt qu'appel direct à Zendesk** : l'API ajoute de la valeur (référence unique, règles métier, traduction des statuts) et isole les clients des détails de Zendesk.
- **Base locale** : elle porte la référence métier, la piste d'audit et le lien entre les identifiants de l'API et ceux de Zendesk.
- **File de messages** : elle garantit qu'un courrier enregistré n'est jamais perdu, même si Zendesk est indisponible ou limite les appels.

3. Cartographie des objets de données

3.1 Base locale (PostgreSQL)

Table courriers

Champ	Type	Description
id	Long	Identifiant interne
reference	String	Référence métier unique (exemple : ARR-2026-00123)
type	Enum	arrivant, depart, interne
expediteur / destinataire	String	Nom, entreprise, e-mail, adresse
date_reception / date_envoi	Date	Date de réception ou d'envoi
date_document	Date	Date inscrite sur le document
objet	String	Objet du courrier
urgence	Enum	normal, urgent, tres_urgent
confidentialite	Enum	public, confidentiel
statut	Enum	nouveau, en_cours_instruction, en_attente_validation, repondu, cloture
service	String	Service compétent
zendesk_ticket_id	Long	Lien vers le ticket Zendesk

statut_synchronisation	Enum	synchronise, en_attente, erreur

Le champ zendesk_ticket_id est indispensable : quand Zendesk notifie un changement, il parle du ticket 4587 et ne connaît pas la référence ARR-2026-00123. L'API conserve donc l'identifiant de l'autre système pour retrouver le bon courrier.

Table pieces_jointes

Champ	Type	Description
id	Long	Identifiant de la pièce jointe (piece_id)
courrier_id	Long	Courrier auquel elle appartient
nom_fichier	String	Nom du fichier
type_fichier	Enum	PDF, PNG, JPEG
chemin_stockage	String	Emplacement du fichier chiffré

Table journal_audit (insertion seule : aucune modification ni suppression)

Champ	Type	Description
courrier_id	Long	Courrier concerné
date_heure	DateTime	Moment de l'action
acteur	String	Utilisateur ou système (webhook Zendesk) à l'origine de l'action
action	String	Enregistrement, changement de statut, ajout de pièce jointe
ancien_statut, nouveau_statut	Enum	Transition effectuée, le cas échéant

Redis complète la base locale : cache des consultations du registre et compteurs de quotas d'appels vers Zendesk.

3.2 Correspondance avec les objets Zendesk

Entité du projet	Objet Zendesk	Champ	Type	Description

Expéditeur / destinataire	User	id	Long	Identifiant unique Zendesk
Expéditeur / destinataire	User	external_id	String	Identifiant côté API Courriers
Expéditeur / destinataire	User	name	String	Nom complet
Expéditeur / destinataire	User	email / phone	String	Coordonnées de contact
Service / agence	Organization	id	Long	Identifiant unique du service
Service / agence	Organization	name	String	Nom du service ou de l'agence
Service / agence	Organization	group_id	Long	Groupe d'agents chargé du traitement
Courrier	Ticket	id	Long	Numéro de ticket généré
Courrier	Ticket	subject	String	Référence et objet du courrier
Courrier	Ticket	comment	Object	Description et pièces jointes
Courrier	Ticket	status	Enum	new, open, pending, hold, solved, closed
Courrier	Ticket	priority	Enum	low, normal, high, urgent
Courrier	Ticket	type	Enum	question, incident, problem, task (task pour un courrier)

Correspondance entre l'urgence du courrier et la priorité du ticket :

Urgence du courrier	priority Zendesk

Normal	normal
Urgent	high
Très urgent	urgent

3.3 Champs personnalisés du ticket

Zendesk n'impose aucune nomenclature : l'administrateur crée les champs (menu « Ticket Fields ») et chacun reçoit un **identifiant numérique propre à chaque compte**. Ces identifiants se relèvent avec GET /api/v2/ticket_fields.json et se placent en **configuration**, jamais en dur dans le code.

Donnée	Nom logique	Type	Description / valeurs
Référence du courrier	reference_courrier	Texte	Exemple : ARR-2026-00123
Type de courrier	type_courrier	Liste déroulante	arrivant, depart, interne
Confidentialité	confidentialite	Liste déroulante	public, confidentiel
Code agence d'enregistrement	code_agence	Texte	Agence ayant numérisé et enregistré le courrier
Échéance de traitement	echeance_sla	Date et heure	Échéance calculée selon la priorité
Référence de dossier client	reference_dossier	Texte	Facultatif (numéro de compte, par exemple)

4. Spécifications des interfaces et contrats d'API

4.1 Conventions

- Chemin de base : /api/v1. Les URL ci-dessous sont données sans ce préfixe.
- Une URL désigne une **ressource** (un nom), jamais une action.
- Les noms de ressources sont au **pluriel**, en minuscules, avec des tirets (pieces-jointes).

- Le **verbe HTTP** exprime l'action : POST crée, GET lit, PATCH modifie partiellement.
- Une ressource enfant se place après l'identifiant de son parent.
- Les filtres passent en **paramètres de requête** (après le ?).
- Les échanges se font en JSON (sauf l'envoi de fichiers, en multipart/form-data).
- L'API est **stateless** : chaque requête contient tout ce qu'il faut pour être traitée.

4.2 API Courriers : endpoints

Besoin du cahier	Endpoint	Section du cahier
Enregistrer un courrier	POST /courriers	3.1
Consulter un courrier	GET /courriers/{id}	3.4
Consulter / filtrer le registre	GET /courriers?statut=...&service=...&expediteur=...&date_debut=...&date_fin=...	3.4
Changer le statut (dont valider)	PATCH /courriers/{id}/statut	3.2, 3.3
Ajouter une pièce jointe	POST /courriers/{id}/pieces-jointes	3.1
Lister les pièces jointes	GET /courriers/{id}/pieces-jointes	3.1

Télécharger une pièce jointe	GET /courriers/{courrier_id}/pieces-jointes/{piece_id}	3.1
Consulter l'historique d'un courrier	GET /courriers/{id}/historique	3.4 (traçabilité)
Recevoir les notifications Zendesk	POST /webhooks/zendesk	4.2

POST /courriers : enregistrer un courrier

1. L'API reçoit les données : expéditeur / destinataire, date de réception ou d'envoi, date du document, objet, urgence (Normal, Urgent, Très urgent), confidentialité (Public, Confidentiel).
2. Elle génère la **référence unique** (par exemple ARR-2026-00123 ou DEP-2026-00045, format configurable).
3. Elle enregistre le courrier dans la base et écrit la première ligne du journal d'audit.
4. Elle publie un message dans la file : un traitement crée le ticket Zendesk (section 4.3) et conserve son numéro dans zendesk_ticket_id.
5. Elle répond 201 Created avec le courrier et sa référence. Si Zendesk n'a pas encore pu créer le ticket, statut_synchronisation vaut en_attente jusqu'au retour du service.

GET /courriers : consulter le registre

Le registre et son filtrage forment **un seul endpoint** : sans paramètre, il renvoie tout le registre chronologique ; chaque paramètre restreint la liste.

Critère du cahier	Paramètre
Période	date_debut, date_fin

Statut	statut
Expéditeur	expediteur
Service	service

PATCH /courriers/{id}/statut : changer le statut

Corps de la requête : {"statut": "en_attente_validation"} (exemple).

- Il n'existe **pas d'endpoint /valider** : valider un projet de réponse, c'est changer son statut (transitions autorisées : section 5.2).
- L'API met à jour sa base, écrit le journal d'audit, puis met à jour le ticket Zendesk pour garder les deux systèmes synchronisés.
- Seul le **validateur / responsable d'unité** peut effectuer la transition de validation (section 6.2).

Pièces jointes

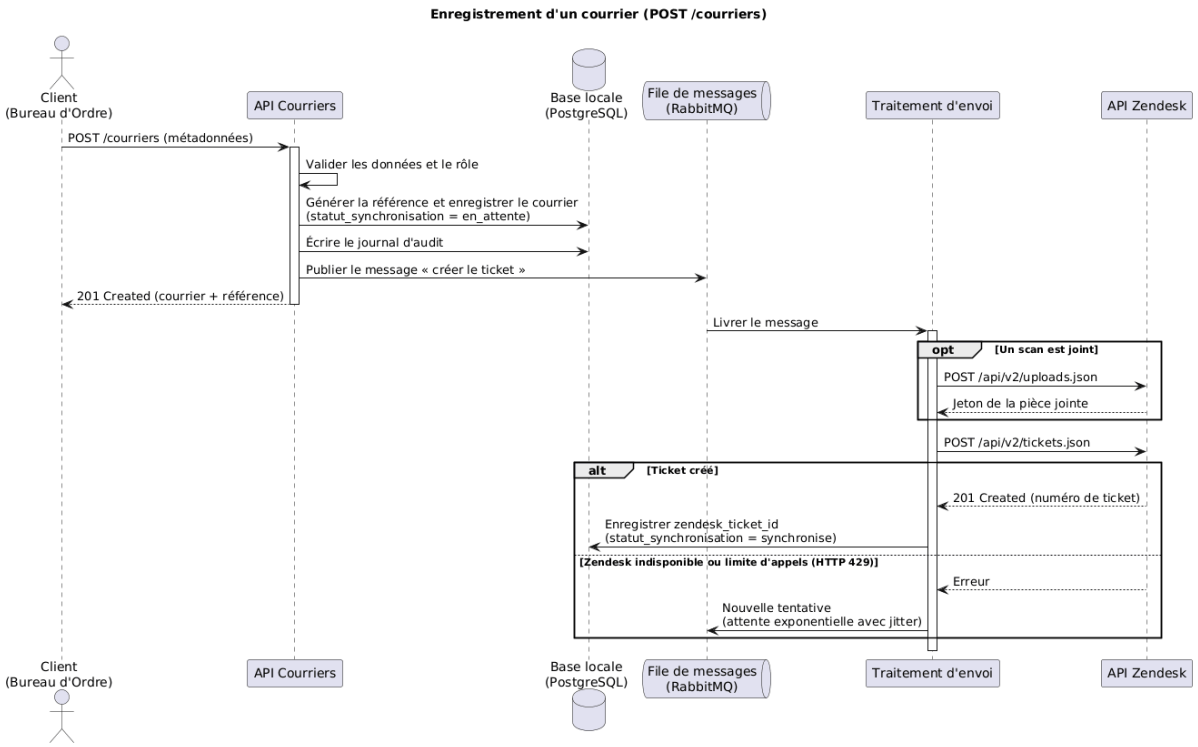
- L'envoi se fait par POST avec le fichier dans le corps de la requête (multipart/form-data).
- Le type du fichier n'apparaît pas dans l'URL : l'API **vérifie** qu'il s'agit d'un PDF, PNG ou JPEG et rejette le reste.
- Le fichier est ensuite transmis à Zendesk pour être attaché au ticket.

Codes de réponse

Code	Cas d'usage
200 OK	Lecture ou modification réussie
201 Created	Courrier ou pièce jointe créé
400 Bad Request	Données invalides (champ manquant, type de fichier refusé)
401 Unauthorized	Authentification absente ou invalide
403 Forbidden	Rôle insuffisant (par exemple validation par un agent traitant)
404 Not Found	Courrier ou pièce jointe inexistant

409 Conflict	Transition de statut interdite (par exemple Nouveau vers Répondu)

Diagramme de séquence de l'enregistrement d'un courrier



4.3 Ingestion d'un ticket dans Zendesk

Chaque nouveau courrier génère un ticket dédié via POST /api/v2/tickets.json. Le scan est d'abord envoyé à Zendesk (/api/v2/uploads.json), qui renvoie un jeton à joindre au commentaire du ticket. Exemple de requête envoyée par l'API (identifiants fictifs) :

```
{
  "ticket": {
        "subject": "ARR-2026-00123 : Demande de remboursement",
        "comment": {
        "body": "Courrier arrivant reçu le 22/10/2026, à instruire.",
        "public": false,
        "uploads": ["<jeton_de_la_pièce_jointe>"]
        },
        "priority": "high",
```

```
        "type": "task",
        "requester_id": 987654321,
        "organization_id": 12345,
        "custom_fields": [
        {"id": 100000001, "value": "ARR-2026-00123"},
        {"id": 100000002, "value": "arrivant"},
        {"id": 100000003, "value": "confidentiel"}
        ],
        "tags": ["courrier_arrivant", "service_juridique"]
  }
}
```

À la réponse, l'API enregistre le numéro du ticket dans zendesk_ticket_id et passe statut_synchronisation à synchronise.

4.4 Consommation des webhooks Zendesk

Si un agent change un ticket directement dans Zendesk, l'API ne le saurait pas sans mécanisme particulier. Le **webhook** inverse le sens de l'appel : c'est Zendesk qui prévient l'API en temps réel, ce qui évite de l'interroger en permanence.

Endpoint récepteur : POST /webhooks/zendesk, exposé en HTTPS sur une adresse **publique** (Zendesk est un service cloud et ne peut pas joindre une adresse purement interne). Cet endpoint est destiné à Zendesk, non aux utilisateurs de l'API.

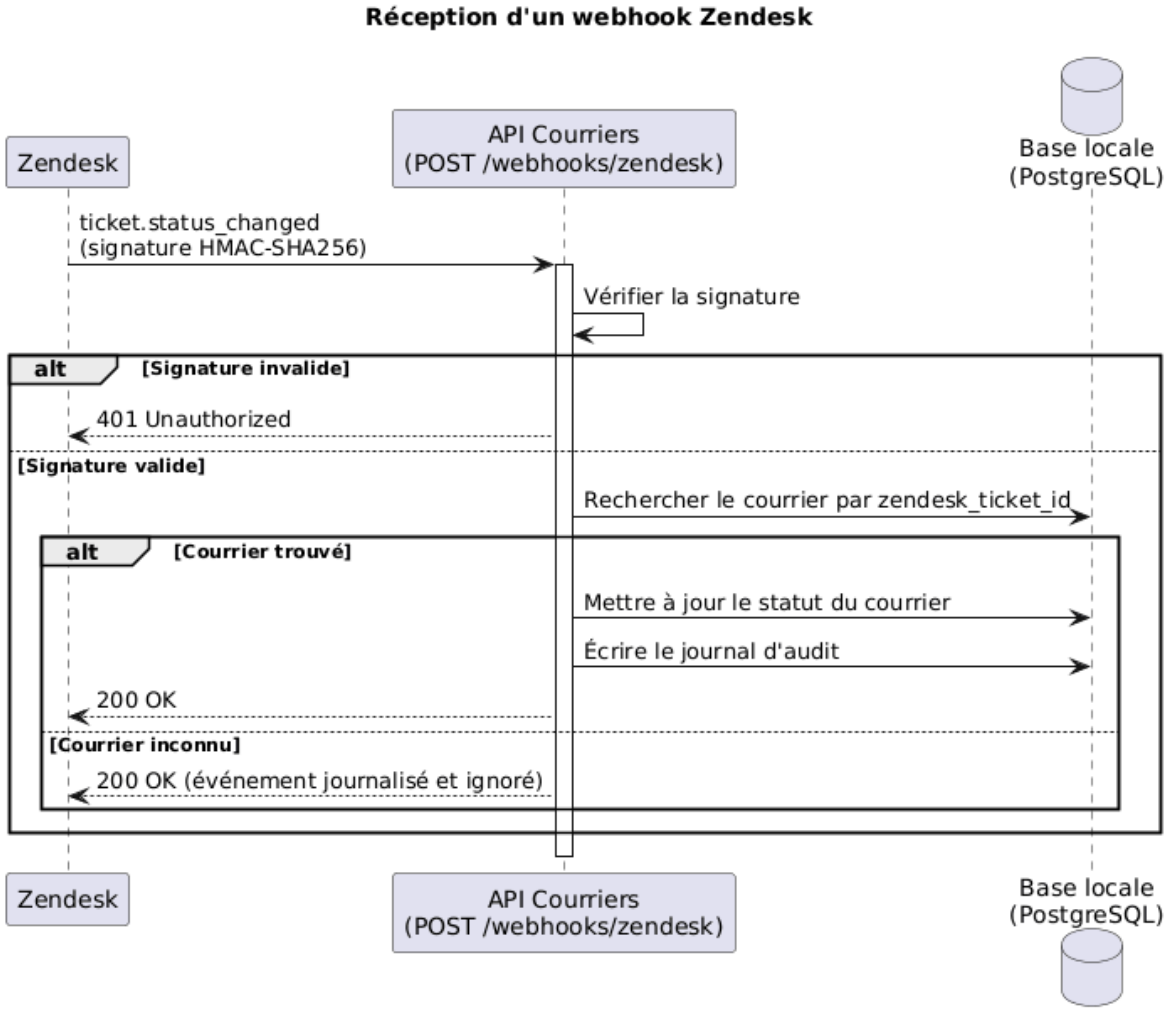
Événement Zendesk	Traitement par l'API
zen:event-type:ticket.created	Confirmer la création du ticket et écrire la piste d'audit locale
zen:event-type:ticket.status_changed	Mettre à jour le statut du courrier et écrire le journal d'audit

Traitement à la réception :

1. **Vérifier la signature** : Zendesk signe chaque envoi en HMAC-SHA256 (en-têtes X-Zendesk-Webhook-Signature et X-Zendesk-Webhook-Signature-Timestamp) ; toute requête mal signée est rejetée.
2. Retrouver le courrier grâce à zendesk_ticket_id.

3. Mettre à jour son statut dans la base et écrire le journal d'audit.

Diagramme de séquence : webhook Zendesk

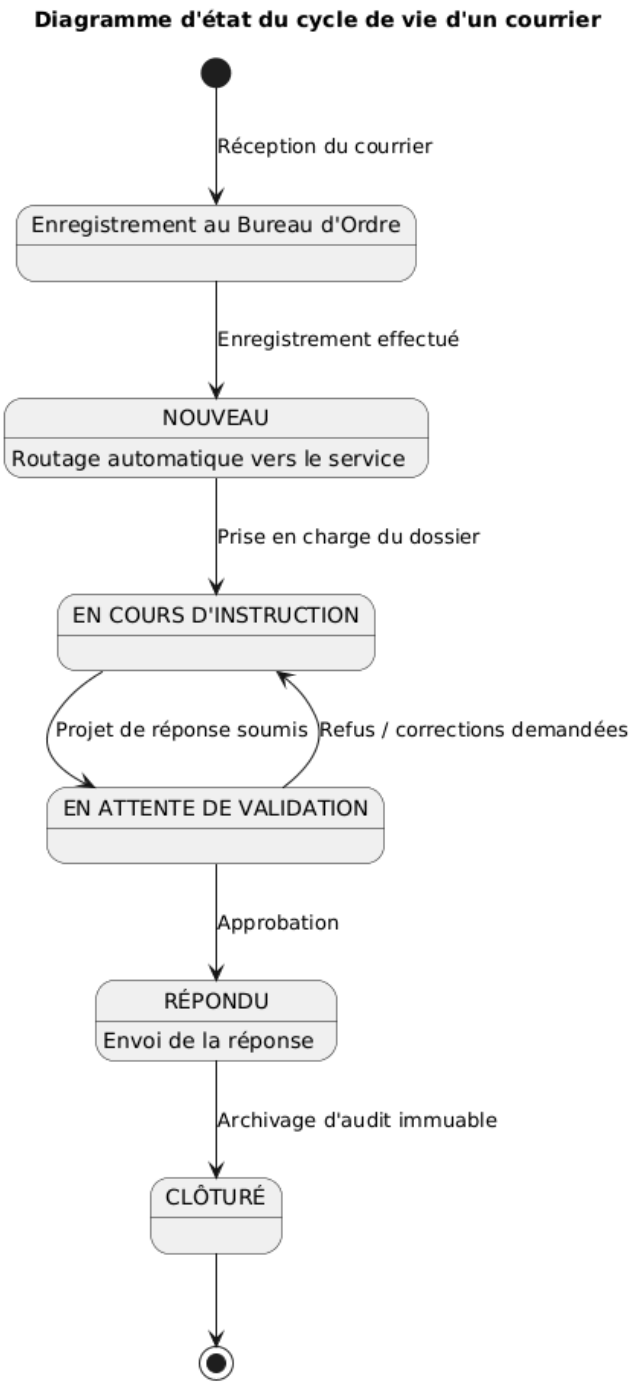


4.5 Affectation automatique et SLA

Le routage vers le service compétent (juridique, RH, direction) et les délais de réponse selon la priorité sont configurés **dans Zendesk** (déclencheurs et SLA) et livrés sous forme de scripts de configuration. L'alerte préventive avant dépassement d'un délai, destinée à l'escalade vers le chef de service, repose sur une automatisation Zendesk qui notifie l'API.

5. Workflows et diagramme des états

5.1 Cycle de vie d'un courrier



5.2 Transitions autorisées

L'API n'accepte que les transitions du tableau ci-dessous et répond 409 Conflict pour toute autre.

De	Vers	Déclenché par	Acteur autorisé
(enregistrement)	Nouveau	Enregistrement du courrier, routage automatique vers le service	Bureau d'Ordre
Nouveau	En cours d'instruction	Prise en charge du dossier	Agent traitant
En cours d'instruction	En attente de validation	Soumission du projet de réponse	Agent traitant
En attente de validation	Répondu	Approbation de la réponse (la réponse part ensuite)	Validateur
En attente de validation	En cours d'instruction	Refus, retour à l'agent pour correction	Validateur
Répondu	Clôturé	Clôture et archivage d'audit immuable	Zendesk (automatique)

5.3 Correspondance avec les statuts Zendesk

Les statuts du courrier sont ceux du métier ; Zendesk définit six statuts standard (new, open, pending, hold, solved, closed). L'API effectue une **traduction** entre les deux :

Statut du courrier	Statut du ticket Zendesk
Nouveau	new
En cours d'instruction	open
En attente de validation	hold
Répondu	solved
Clôturé	closed

- **hold** correspond à l'attente d'un interlocuteur interne (le validateur) ; il doit être activé par l'administrateur Zendesk. pending, qui correspond à l'attente du demandeur, n'est pas retenu : il repasse automatiquement à open dès que le demandeur répond, ce qui fausserait la synchronisation.

- **closed** est appliqué par Zendesk : l'API vise solved, puis reçoit la clôture par webhook et passe le courrier à Clôturé.

6. Sécurité, confidentialité et résilience

6.1 Authentification

- **Clients vers l'API Courriers** : protocole OAuth 2.0, avec restriction des adresses IP autorisées (liste blanche).
- **API Courriers vers Zendesk** : token d'API Zendesk, transmis dans l'en-tête Authorization: Basic avec l'encodage base64 de adresse_email/token:jeton_api.
- Les jetons et identifiants ne figurent **jamais dans le code source** : ils sont fournis par configuration (variables d'environnement).

6.2 Contrôle d'accès (RBAC)

Action	Bureau d'Ordre	Agent traitant	Validateur	Administrateur
Enregistrer un courrier	Oui	Non	Non	Oui
Consulter le registre	Oui	Oui	Oui	Oui
Changer le statut	Non	Oui (sauf validation)	Oui	Oui
Valider un projet de réponse	Non	Non	**Oui**	Oui
Paramétrer accès et workflows	Non	Non	Non	Oui

6.3 Confidentialité et protection des données

- Les courriers **confidentiels** sont restreints à certains groupes d'utilisateurs, et les champs sensibles sont masqués pour les rôles non habilités : seuls les champs nécessaires au traitement du courrier leur sont rendus visibles.
- Chiffrement en transit : **TLS 1.3**.
- Chiffrement des pièces jointes **au repos**.

- Vérification de la signature de chaque requête reçue sur le webhook (section 4.4).
- Journal d'audit **immuable** : toute action sur un courrier y est tracée.

6.4 Résilience

Risque	Mécanisme
Zendesk limite les appels (réponse HTTP 429)	Nouvelles tentatives avec **attente exponentielle et aléa (exponential backoff with jitter)**, en respectant le délai indiqué par Zendesk
Zendesk est indisponible	**Circuit breaker** (Resilience4j) : l'API cesse d'appeler Zendesk le temps de la panne
Courrier enregistré pendant une panne	Le message reste dans la **file RabbitMQ** et le courrier garde statut_synchronisation = en_attente ; le ticket est créé dès le retour de Zendesk

Ainsi, aucun courrier n'est perdu et l'enregistrement au Bureau d'Ordre n'est pas interrompu par une panne de Zendesk.

7. Tests, documentation et déploiement

7.1 Stratégie de tests

- **Tests unitaires** avec simulation (mocks) de l'API Zendesk.
- **Tests d'intégration** : création du courrier, création du ticket, pièces jointes, synchronisation des statuts par webhook, clôture.
- **Tests de charge**, notamment sur le comportement face aux limites d'appels de Zendesk.
- **Cahier de recette** validant la création de courrier, la synchronisation du ticket, la gestion des pièces jointes et la clôture.

7.2 Documentation

- Description de l'API au format **OpenAPI / Swagger**.
- Guide d'installation et d'administration, guide utilisateur (Bureau d'Ordre et agents traitants).

- **Scripts de configuration Zendesk** : champs personnalisés, déclencheurs, SLA et webhook.

7.3 Déploiement

Le déploiement est **conteneurisé** : un conteneur par composant (API Courriers, PostgreSQL, Redis, RabbitMQ), décrits dans un fichier de composition. Un guide de déploiement conteneurisé est livré avec le code source et le dossier d'architecture.

CAHIER D'IMPLÉMENTATION

Gestion des courriers d'une agence(GEC-AGENCE)

via l'API Zendesk (REST v2)

1. Introduction

Ce cahier d'implémentation est le quatrième livrable du projet de gestion des courriers d'une agence. Il fait suite au cahier des charges (le quoi), au dossier d'analyse (le métier) et au dossier de conception (l'architecture). Il décrit comment la solution a été réalisée : outils retenus, organisation du code, rôle de chaque fichier, configuration, procédure de lancement, plan de recette et difficultés rencontrées.

Le principe directeur est de modéliser chaque courrier comme un ticket Zendesk. L'application ne réinvente pas un moteur de suivi : elle s'appuie sur l'API REST de Zendesk (tickets, commentaires, recherche) et la traduit en vocabulaire métier (courrier, référence, note de traitement).

1.1 Objectifs de l'implémentation

- Traduire chaque besoin fonctionnel (BF1 à BF10) en code exécutable.
- Respecter l'architecture en trois niveaux répartis : navigateur, backend passerelle, service SaaS distant.
- Isoler Zendesk derrière un adaptateur pour permettre un mode simulation sans compte.
- Appliquer la sécurité par conception : le jeton Zendesk ne quitte jamais le serveur.

2. Environnement et outils

Chaque outil est présenté ici avant que son code n'apparaisse dans la suite du document.

Outil	Version	Rôle dans le projet
Node.js	18 ou plus	Environnement d'exécution JavaScript côté serveur ; fournit aussi la fonction fetch native utilisée pour appeler Zendesk.
npm	inclus avec Node	Gestionnaire de dépendances et lanceur de scripts (npm start).
Express	^4.19.2	Framework web : routage HTTP, analyse du JSON, service des fichiers statiques.
Helmet	^7.1.0	Ajoute des en-têtes HTTP de sécurité (CSP, X-Content-Type-Options, etc.).
express-rate-limit	^7.4.0	Limite le débit : 100 requêtes par 15 minutes et par adresse IP.
dotenv	^16.4.5	Charge les variables d'environnement depuis le fichier .env.
API REST Zendesk	v2	Service distant : tickets, commentaires, recherche, comptage.
HTML / CSS / JavaScript	natif	Frontend sans framework : un fichier HTML, une feuille de style, un script.

Aucun framework frontend n'est imposé. Ce choix réduit les dépendances et illustre le dialogue direct entre un client léger et une API REST.

3. Structure du projet

Le code est organisé en couches, conformément au dossier de conception. Chaque couche n'appelle que celle située juste en dessous.

Couche	Fichier	Responsabilité
Présentation	public/index.html, style.css, app.js	Interface utilisateur, appels fetch vers /api.
API	src/routes/courriers.js	Routes HTTP, validation des entrées.
Service / Mapping	src/mapper.js	Traduction métier ↔ Zendesk.
Infrastructure	src/zendeskClient.js, zendeskMock.js	Appels HTTP, authentification, relance ; version simulée.
Transverse	src/server.js, src/errors.js	Sécurité, erreurs, fichiers statiques, démarrage.

Arborescence complète :

```
courriers-zendesk/
```

```
├── package.json
├── .env.example
├── .gitignore
├── README.md
├── src/
│   ├── server.js
│   ├── errors.js
│   ├── zendeskClient.js
│   ├── zendeskMock.js
│   ├── mapper.js
│   └── routes/courriers.js
└── public/
      ├── index.html
      ├── style.css
      └── app.js
```

4. Implémentation du backend

4.1 Dépendances et scripts (package.json)

Le fichier package.json déclare les quatre dépendances, la version minimale de Node et deux scripts : npm start lance le serveur ; npm run dev le relance automatiquement à chaque modification grâce à l'option node --watch.

Fichier : package.json

```
{
  "name": "courriers-zendesk",
  "version": "1.0.0",
  "main": "src/server.js",
  "scripts": { "start": "node src/server.js", "dev": "node --watch src/server.js" },
  "engines": { "node": ">=18" },
  "dependencies": {
       "dotenv": "^16.4.5",
       "express": "^4.19.2",
       "express-rate-limit": "^7.4.0",
    "helmet": "^7.1.0"
  }
}
```

4.2 Gestion des erreurs (errors.js)

Ce module centralise la gestion des erreurs. HttpError porte un code HTTP et des détails ; asyncH enveloppe les routes asynchrones pour que toute exception atteigne le middleware ; errorMiddleware renvoie une réponse JSON normalisée. Une erreur venant de Zendesk avec un code 5xx est convertie en 502 (Bad Gateway), ce qui signale clairement au client que le service distant est en cause.

Fichier : src/errors.js

```
class HttpError extends Error {
  constructor(status, message, details) {
        super(message);
        this.status = status;
        this.details = details;
  }
}
const asyncH = (fn) => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);

function errorMiddleware(err, req, res, next) {
  const status = err.status || 500;
  // Erreurs venant de Zendesk (>=500 ou réseau) => 502
  const out = status >= 500 && err.fromZendesk ? 502 : status;
  if (out >= 500) console.error(err);
  res.status(out).json({ erreur: err.message || 'Erreur interne', details: err.details });
}
module.exports = { HttpError, asyncH, errorMiddleware };
```

4.3 Client Zendesk (zendeskClient.js)

Cet adaptateur est le seul fichier qui connaît le protocole Zendesk. Trois points sont essentiels :

- **Authentification :** l'authentification : l'en-tête Authorization est « Basic » suivi de la valeur Base64 de « email/token:JETON ». Le jeton est lu dans l'environnement et n'est jamais exposé au navigateur ;
- **Résilience :** lorsque Zendesk répond 429, le client attend la durée indiquée par l'en-tête Retry-After puis réessaie, au plus deux fois ;
- **Normalisation des erreurs :** les codes 400, 404, 422 et 429 sont renvoyés tels quels (422 devient 400) ; tout autre échec devient 502.

Au début du fichier, si ZENDESK_MODE vaut mock, le module exporte le faux Zendesk de la section suivante et s'arrête : le reste de l'application n'est pas modifié.

Fichier : src/zendeskClient.js

```
// Mode simulation : aucun compte Zendesk nécessaire
if (process.env.ZENDESK_MODE === 'mock') {
```

```
  console.log('Mode SIMULATION Zendesk activé (données en mémoire)');
  module.exports = require('./zendeskMock');
  return;
}

const { HttpError } = require('./errors');

const { ZENDESK_SUBDOMAIN, ZENDESK_EMAIL, ZENDESK_API_TOKEN } = process.env;
if (!ZENDESK_SUBDOMAIN || !ZENDESK_EMAIL || !ZENDESK_API_TOKEN) {
      console.error('Variables ZENDESK_* manquantes (voir .env.example) ou mettez
ZENDESK_MODE=mock');
  process.exit(1);
}

const BASE = `https://${ZENDESK_SUBDOMAIN}.zendesk.com/api/v2`;
// Authentification officielle : email/token:TOKEN encodé en Base64
const AUTH = 'Basic ' + Buffer.from(`ZENDESK_EMAIL/token:
{ZENDESK_API_TOKEN}`).toString('base64');
const sleep = (ms) => new Promise((r) => setTimeout(r, ms));

async function zd(path, { method = 'GET', body, retries = 2 } = {}) {
  let res;
  try {
       res = await fetch(BASE + path, {
       method,
       headers: { Authorization: AUTH, 'Content-Type': 'application/json', Accept:
'application/json' },
       body: body ? JSON.stringify(body) : undefined,
       });
  } catch (e) {
       const err = new HttpError(502, 'Service Zendesk injoignable');
       err.fromZendesk = true;
       throw err;
  }
  // Rate limit : on attend Retry-After puis on réessaie
  if (res.status === 429 && retries > 0) {
       await sleep((Number(res.headers.get('retry-after')) || 1) * 1000);
       return zd(path, { method, body, retries: retries - 1 });
  }
  if (res.status === 204) return null;
  const data = await res.json().catch(() => ({}));
  if (!res.ok) {
       const msg = data.error?.message || data.error || data.description || `Erreur
Zendesk ${res.status}`;
       const status = [400, 404, 422, 429].includes(res.status) ? (res.status === 422 ?
400 : res.status) : 502;
       const err = new HttpError(status, typeof msg === 'string' ? msg :
JSON.stringify(msg), data.details);
       err.fromZendesk = true;
       throw err;
  }
```

```
  return data;
}

module.exports = {
  search: (query, page = 1, perPage = 25) =>
       zd(`/search.json?query=encodeURIComponent(query)&sort_by = created_at&sort_order = desc&page =
{page}&per_page=${perPage}`),
  count: (query) => zd(`/search/count.json?query=${encodeURIComponent(query)}`),
  getTicket: (id) => zd(`/tickets/${id}.json`),
  getComments: (id) => zd(`/tickets/${id}/comments.json`),
  createTicket: (ticket) => zd('/tickets.json', { method: 'POST', body: { ticket } }),
  updateTicket: (id, ticket) => zd(`/tickets/${id}.json`, { method: 'PUT', body: { ticket }
}),
  deleteTicket: (id) => zd(`/tickets/${id}.json`, { method: 'DELETE' }),
};
```

4.4 Mode simulation (zendeskMock.js)

Ce module reproduit en mémoire les sept fonctions exposées par le client réel (search, count, getTicket, getComments, createTicket, updateTicket, deleteTicket). Il interprète les filtres tags: et status: ainsi que la recherche par mot-clé dans l'objet et les commentaires. Il permet de tester et de présenter le projet sans compte Zendesk. Les données sont perdues à l'arrêt du serveur.

Fichier : src/zendeskMock.js

```
// Faux Zendesk en mémoire : même interface que zendeskClient.js
const { HttpError } = require('./errors');
let seq = 1000;
const tickets = new Map();
const comments = new Map();
const now = () => new Date().toISOString();
const low = (s) => String(s).toLowerCase();

function match(t, query) {
  return query.split(/\s+/).filter(Boolean).every((term) => {
       if (term.startsWith('tags:')) return t.tags.includes(term.slice(5));
       if (term.startsWith('status:')) return t.status === term.slice(7);
       if (term.startsWith('type:')) return true;
       return low(t.subject).includes(low(term)) ||
       (comments.get(t.id) || []).some((c) => low(c.body).includes(low(term)));
  });
}
const find = (id) => {
  const t = tickets.get(Number(id));
  if (!t) throw new HttpError(404, 'Ticket introuvable');
  return t;
};
```

```
const addComment = (id, c) => {
  const list = comments.get(id) || [];
    list.push({ id: ++seq, author_id: 1, body: c.body, public: c.public !== false,
created_at: now() });
  comments.set(id, list);
};

module.exports = {
  search: async (query, page = 1, perPage = 25) => {
        const all = [...tickets.values()].filter((t) => match(t, query))
        .sort((a, b) => b.created_at.localeCompare(a.created_at));
        const start = (page - 1) * perPage;
        return { results: all.slice(start, start + perPage), count: all.length,
                next_page: start + perPage < all.length ? 'next' : null };
  },
    count: async (query) => ({ count: [...tickets.values()].filter((t) => match(t,
query)).length }),
  getTicket: async (id) => ({ ticket: find(id) }),
  getComments: async (id) => { find(id); return { comments: comments.get(Number(id)) || []
}; },
  createTicket: async (data) => {
        const id = ++seq;
        const t = { id, subject: data.subject, tags: data.tags || [], priority:
data.priority || 'normal',
                status: 'new', type: data.type, created_at: now(), updated_at: now() };
        tickets.set(id, t);
        if (data.comment) addComment(id, data.comment);
        return { ticket: t };
  },
  updateTicket: async (id, patch) => {
        const t = find(id);
        if (patch.status) t.status = patch.status;
        if (patch.priority) t.priority = patch.priority;
        if (patch.comment) addComment(t.id, patch.comment);
        t.updated_at = now();
        return { ticket: t };
  },
       deleteTicket: async (id) => { find(id); tickets.delete(Number(id));
comments.delete(Number(id)); return null; },
};
```

4.5 Traduction métier ↔ Zendesk (mapper.js)

Le mapper applique la table de correspondance du dossier d'analyse. Un courrier devient un ticket de type task, dont le sujet est préfixé par la référence « [CR-AAAA-XXXXXX] » et dont les tags portent le marqueur courrier, le type (entrant, sortant, interne), la référence et le service. Les statuts métier sont traduits : enregistré = new, en traitement = open, en attente =

pending, traité = solved. La fonction inverse toCourrier reconstitue l'objet métier à partir du ticket.

Fichier : src/mapper.js

```
const TYPES = ['entrant', 'sortant', 'interne'];
const PRIORITES = ['low', 'normal', 'high', 'urgent'];
const STATUT_TO_ZD = { enregistre: 'new', en_traitement: 'open', en_attente: 'pending',
traite: 'solved' };
const ZD_TO_STATUT = { new: 'enregistre', open: 'en_traitement', pending: 'en_attente',
hold: 'en_attente', solved: 'traite', closed: 'cloture' };

const slug = (s) => s.toLowerCase().normalize('NFD').replace(/[\u0300-\u036f]/g,
'').replace(/[^a-z0-9]+/g, '_').replace(/^_|_$/g, '');
const genRef = () => `CR-newDate().getFullYear() −
{Date.now().toString(36).toUpperCase().slice(-6)}`;

function toZendeskTicket({ type, objet, contenu, priorite = 'normal', service, nom, email
}) {
  const reference = genRef();
  const tags = ['courrier', type, `ref_${reference.toLowerCase()}`];
  if (service) tags.push(`svc_${slug(service)}`);
  const ticket = {
       subject: `[${reference}] ${objet}`,
       comment: { body: contenu },
       priority: priorite,
       type: 'task',
       tags,
  };
  if (email) ticket.requester = { name: nom || email, email };
  return ticket;
}

function toCourrier(t) {
  const tags = t.tags || [];
  const ref = (t.subject || '').match(/^\[(CR-[^\]]+)\]\s*(.*)$/);
  return {
       id: t.id,
       reference: ref ? ref[1] : null,
       objet: ref ? ref[2] : t.subject,
       type: tags.find((x) => TYPES.includes(x)) || null,
       service: (tags.find((x) => x.startsWith('svc_')) || '').replace('svc_', '') ||
null,
       priorite: t.priority,
       statut: ZD_TO_STATUT[t.status] || t.status,
       creeLe: t.created_at,
       misAJourLe: t.updated_at,
  };
}

const toNote = (c) => ({ id: c.id, auteurId: c.author_id, texte: c.body, publique:
c.public, creeLe: c.created_at });
```

```
module.exports = { TYPES, PRIORITES, STATUT_TO_ZD, toZendeskTicket, toCourrier, toNote };
```

4.6 Routes de l'API interne (courriers.js)

Ce fichier expose les sept routes décrites dans le dossier de conception. Chaque entrée est validée avant tout appel à Zendesk : type, objet de 3 à 150 caractères, contenu non vide, priorité autorisée, e-mail valide. Le texte de recherche est nettoyé pour éviter l'injection d'opérateurs dans la requête Zendesk. La note de traitement est ajoutée comme commentaire non public (public à false).

Méthode	Route	Description	Codes
GET	/api/courriers	Liste paginée avec filtres type, statut, q	200, 400
GET	/api/courriers/:id	Détail et notes	200, 400, 404
POST	/api/courriers	Création	201, 400
PUT	/api/courriers/:id	Statut et priorité	200, 400, 404
POST	/api/courriers/:id/notes	Ajout d'une note interne	201, 400
DELETE	/api/courriers/:id	Suppression	204, 404
GET	/api/stats	Compteurs par statut et par type	200

Fichier : src/routes/courriers.js

```
const express = require('express');
const zd = require('../zendeskClient');
const { HttpError, asyncH } = require('../errors');
const { TYPES, PRIORITES, STATUT_TO_ZD, toZendeskTicket, toCourrier, toNote } =
require('../mapper');

const router = express.Router();
const emailRe = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
const parseId = (v) => {
  const n = Number(v);
  if (!Number.isInteger(n) || n <= 0) throw new HttpError(400, 'Identifiant invalide');
  return n;
};

function validateCreation(b) {
  const e = [];
  if (!TYPES.includes(b.type)) e.push('type doit être entrant, sortant ou interne');
   if (typeof b.objet !== 'string' || b.objet.trim().length < 3 || b.objet.length > 150)
e.push('objet : 3 à 150 caractères');
  if (typeof b.contenu !== 'string' || !b.contenu.trim()) e.push('contenu obligatoire');
```

```
  if (b.priorite && !PRIORITES.includes(b.priorite)) e.push('priorité invalide');
  if (b.email && !emailRe.test(b.email)) e.push('email invalide');
  if (e.length) throw new HttpError(400, 'Validation échouée', e);
}

// Liste + filtres
router.get('/courriers', asyncH(async (req, res) => {
  const { type, statut, q } = req.query;
  const page = Math.max(1, parseInt(req.query.page) || 1);
  let query = 'type:ticket tags:courrier';
  if (type) {
       if (!TYPES.includes(type)) throw new HttpError(400, 'type invalide');
       query += ` tags:${type}`;
  }
  if (statut) {
       if (!STATUT_TO_ZD[statut]) throw new HttpError(400, 'statut invalide');
       query += ` status:${STATUT_TO_ZD[statut]}`;
  }
  if (q) query += ` ${String(q).replace(/[^\p{L}\p{N}\s\-_.@]/gu, '')}`;
  const data = await zd.search(query, page);
  res.json({
       courriers:    data.results.filter((r)    =>    r.result_type    ===    'ticket'    ||
r.subject).map(toCourrier),
       total: data.count,
    page,
    pageSuivante: !!data.next_page,
  });
}));

// Détail + notes
router.get('/courriers/:id', asyncH(async (req, res) => {
  const id = parseId(req.params.id);
  const [t, c] = await Promise.all([zd.getTicket(id), zd.getComments(id)]);
  res.json({ ...toCourrier(t.ticket), notes: c.comments.map(toNote) });
}));

// Création
router.post('/courriers', asyncH(async (req, res) => {
  validateCreation(req.body);
   const b = { ...req.body, objet: req.body.objet.trim(), contenu: req.body.contenu.trim()
};
  const created = await zd.createTicket(toZendeskTicket(b));
  res.status(201).json(toCourrier(created.ticket));
}));

// Mise à jour statut / priorité
router.put('/courriers/:id', asyncH(async (req, res) => {
  const id = parseId(req.params.id);
  const { statut, priorite } = req.body;
  const patch = {};
  if (statut !== undefined) {
```

```
        if (!STATUT_TO_ZD[statut]) throw new HttpError(400, 'statut invalide');
        patch.status = STATUT_TO_ZD[statut];
  }
  if (priorite !== undefined) {
        if (!PRIORITES.includes(priorite)) throw new HttpError(400, 'priorité invalide');
        patch.priority = priorite;
  }
  if (!Object.keys(patch).length) throw new HttpError(400, 'Rien à mettre à jour');
  const r = await zd.updateTicket(id, patch);
  res.json(toCourrier(r.ticket));
}));

// Ajout d'une note interne
router.post('/courriers/:id/notes', asyncH(async (req, res) => {
  const id = parseId(req.params.id);
  const texte = (req.body.texte || '').trim();
  if (!texte) throw new HttpError(400, 'texte obligatoire');
  await zd.updateTicket(id, { comment: { body: texte, public: false } });
  res.status(201).json({ ok: true });
}));

// Suppression
router.delete('/courriers/:id', asyncH(async (req, res) => {
  await zd.deleteTicket(parseId(req.params.id));
  res.status(204).end();
}));

// Statistiques
router.get('/stats', asyncH(async (req, res) => {
  const base = 'type:ticket tags:courrier';
  const statuts = Object.keys(STATUT_TO_ZD);
  const [total, ...rest] = await Promise.all([
        zd.count(base),
        ...statuts.map((s) => zd.count(`basestatus:{STATUT_TO_ZD[s]}`)),
        ...TYPES.map((t) => zd.count(`basetags:{t}`)),
  ]);
  const parStatut = {}, parType = {};
  statuts.forEach((s, i) => (parStatut[s] = rest[i].count));
  TYPES.forEach((t, i) => (parType[t] = rest[statuts.length + i].count));
  res.json({ total: total.count, parStatut, parType });
}));

module.exports = router;
```

4.7 Serveur (server.js)

Le point d'entrée assemble les composants : chargement de dotenv, Helmet, analyse JSON limitée à 100 Ko, limitation de débit sur /api, montage des routes, réponse 404 JSON pour

toute route /api inconnue, service des fichiers statiques du dossier public, puis middleware d'erreurs en dernier.

Fichier : src/server.js

```
require('dotenv').config();
const path = require('path');
const express = require('express');
const helmet = require('helmet');
const rateLimit = require('express-rate-limit');
const courriers = require('./routes/courriers');
const { errorMiddleware } = require('./errors');

const app = express();
app.use(helmet());
app.use(express.json({ limit: '100kb' }));
app.use('/api', rateLimit({ windowMs: 15 * 60 * 1000, max: 100, standardHeaders: true,
legacyHeaders: false }));
app.use('/api', courriers);
app.use('/api', (req, res) => res.status(404).json({ erreur: 'Route inconnue' }));
app.use(express.static(path.join(__dirname, '..', 'public')));
app.use(errorMiddleware);

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Serveur sur http://localhost:${PORT}`));
```

5. Implémentation du frontend

Le frontend est une page unique servie par Express. Il ne contient aucun secret et communique uniquement avec /api. Toute donnée affichée est échappée par la fonction esc pour prévenir les failles XSS.

5.1 Structure de la page (index.html)

La page comporte quatre zones : cartes de statistiques, formulaire de création, tableau des courriers avec filtres et pagination, et une boîte de dialogue de détail (élément HTML dialog) pour changer le statut, ajouter une note ou supprimer.

Fichier : public/index.html

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Gestion des courriers</title>
  <link rel="stylesheet" href="style.css">
```

```
</head>
<body>
  <header><h1>📮 Gestion des courriers</h1></header>
  <main>
        <section id="stats" class="cards"></section>

        <section class="panel">
        <h2>Nouveau courrier</h2>
        <form id="form">
        <select name="type" required>
        <option value="entrant">Entrant</option>
        <option value="sortant">Sortant</option>
           <option value="interne">Interne</option>
         </select>
        <input name="objet" placeholder="Objet" required minlength="3" maxlength="150">
        <textarea name="contenu" placeholder="Contenu / description" required></textarea>
        <input name="nom" placeholder="Expéditeur / destinataire">
         <input name="email" type="email" placeholder="E-mail (optionnel)">
        <input name="service" placeholder="Service concerné">
        <select name="priorite">
        <option value="normal">Priorité normale</option>
        <option value="low">Basse</option>
        <option value="high">Haute</option>
        <option value="urgent">Urgente</option>
        </select>
        <button type="submit">Enregistrer</button>
        </form>
        <p id="msg"></p>
        </section>

        <section class="panel">
       <h2>Courriers</h2>
        <div class="filters">
        <input id="q" placeholder="Rechercher…">
        <select        id="fType"><option        value="">Tous        types</option><option
value="entrant">Entrant</option><option             value="sortant">Sortant</option><option
value="interne">Interne</option></select>
        <select       id="fStatut"><option       value="">Tous       statuts</option><option
value="enregistre">Enregistré</option><option                      value="en_traitement">En
traitement</option><option          value="en_attente">En          attente</option><option
value="traite">Traité</option></select>
         <button id="btnFiltrer">Filtrer</button>
        </div>
        <div class="scroll">
        <table>

<thead><tr><th>Réf.</th><th>Objet</th><th>Type</th><th>Priorité</th><th>Statut</th><th>Date
</th></tr></thead>
        <tbody id="liste"></tbody>
        </table>
        </div>
```

```
      <div class="pager"><button id="prev">◂</button><span id="pageNum">1</span><button
id="next">▸</button></div>
      </section>
  </main>

  <dialog id="detail">
      <div id="detailBody"></div>
      <button id="fermer">Fermer</button>
  </dialog>
  <script src="app.js"></script>
</body>
</html>
```

5.2 Style (style.css)

La feuille de style est responsive : le formulaire passe sur deux colonnes à partir de 700 pixels, le tableau défile horizontalement sur mobile.

Fichier : public/style.css

```
*{box-sizing:border-box}
body{margin:0;font-family:system-ui,sans-serif;background:#f3f5f9;color:#1f2937}
header{background:#1e3a8a;color:#fff;padding:12px 20px}
h1{margin:0;font-size:1.3rem}
main{max-width:1000px;margin:auto;padding:12px}
.cards{display:grid;grid-template-columns:repeat(auto-fit,minmax(120px,1fr));gap:8px;margin
-bottom:12px}
.card{background:#fff;border-radius:8px;padding:10px;text-align:center;box-shadow:0 1px 3px
#0002}
.card b{display:block;font-size:1.5rem;color:#1e3a8a}
.panel{background:#fff;border-radius:8px;padding:14px;margin-bottom:12px;box-shadow:0 1px
3px #0002}
form{display:grid;gap:8px}
input,select,textarea,button{padding:8px;border:1px solid
#cbd5e1;border-radius:6px;font:inherit}
textarea{min-height:70px}
button{background:#1e3a8a;color:#fff;border:0;cursor:pointer}
button.danger{background:#b91c1c}
.filters{display:flex;gap:6px;flex-wrap:wrap;margin-bottom:8px}
.filters input{flex:1;min-width:140px}
.scroll{overflow-x:auto}
table{width:100%;border-collapse:collapse;font-size:.9rem}
th,td{padding:8px;border-bottom:1px solid #e5e7eb;text-align:left}
tbody tr{cursor:pointer}tbody tr:hover{background:#eef2ff}
.pager{display:flex;gap:12px;justify-content:center;align-items:center;margin-top:8px}
.badge{padding:2px 8px;border-radius:10px;background:#e0e7ff;font-size:.8rem}
.urgent{background:#fecaca}.high{background:#fed7aa}
dialog{max-width:600px;width:92%;border:0;border-radius:10px;padding:16px}
.note{background:#f1f5f9;border-radius:6px;padding:6px;margin:6px 0;font-size:.9rem}
```

```
#msg{margin:6px 0 0;font-weight:600}.ok{color:#15803d}.ko{color:#b91c1c}
@media(min-width:700px){form{grid-template-columns:1fr 1fr}textarea,form
button{grid-column:1/-1}}
```

5.3 Logique (app.js)

Le script utilise l'API fetch. La fonction api centralise les appels et transforme les erreurs du backend en messages lisibles. loadStats et loadList alimentent le tableau de bord et la liste ; openDetail affiche le dialogue ; le formulaire déclenche un rafraîchissement différé de 1,5 seconde après création, car l'index de recherche Zendesk peut avoir un léger retard.

Fichier : public/app.js

```
const $ = (s) => document.querySelector(s);
const esc = (s) => String(s ?? '').replace(/[&<>"']/g, (c) => ({ '&': '&amp;', '<': '&lt;',
'>': '&gt;', '"': '&quot;', "'": '&#39;' }[c]));
const LABELS = { enregistre: 'Enregistré', en_traitement: 'En traitement', en_attente: 'En
attente', traite: 'Traité', cloture: 'Clôturé' };
let page = 1, hasNext = false;

async function api(url, opts = {}) {
  const r = await fetch('/api' + url, { headers: { 'Content-Type': 'application/json' },
...opts, body: opts.body ? JSON.stringify(opts.body) : undefined });
  if (r.status === 204) return null;
  const d = await r.json().catch(() => ({}));
  if (!r.ok) throw new Error([d.erreur, ...(d.details || [])].filter(Boolean).join(' - ')
|| 'Erreur');
  return d;
}
function say(text, ok = true) { const m = $('#msg'); m.textContent = text; m.className = ok
? 'ok' : 'ko'; }

async function loadStats() {
  try {
      const s = await api('/stats');
      $('#stats').innerHTML =
      `<div class="card"><b>${s.total}</b>Total</div>` +
      Object.entries(s.parStatut).map(([k, v]) => `<div class="card"><b>v</b>
{LABELS[k]}</div>`).join('') +
      Object.entries(s.parType).map(([k, v]) => `<div class="card"><b>v</b>
{esc(k)}</div>`).join('');
  } catch (e) { $('#stats').textContent = e.message; }
}

async function loadList() {
  const p = new URLSearchParams({ page });
  if ($('#q').value) p.set('q', $('#q').value);
  if ($('#fType').value) p.set('type', $('#fType').value);
  if ($('#fStatut').value) p.set('statut', $('#fStatut').value);
```

```
  try {
       const d = await api('/courriers?' + p);
    hasNext = d.pageSuivante;
    $('#pageNum').textContent = page;
    $('#liste').innerHTML = d.courriers.map((c) => `
       <tr data-id="${c.id}">
        <td>esc(c.reference) < /td >< td >{esc(c.objet)}</td><td>${esc(c.type)}</td>
       <td><span class="badge esc(c.priorite)" >{esc(c.priorite)}</span></td>
       <td>${esc(LABELS[c.statut] || c.statut)}</td>
       <td>${new Date(c.creeLe).toLocaleDateString('fr-FR')}</td></tr>`).join('')
       || '<tr><td colspan="6">Aucun courrier</td></tr>';
  } catch (e) { ('#liste').innerHTML = ` < tr >< tdcolspan = "6" >{esc(e.message)}</td></tr>`; }
}

async function openDetail(id) {
  const c = await api('/courriers/' + id);
  $('#detailBody').innerHTML = `
       <h3>${esc(c.reference)} – ${esc(c.objet)}</h3>
       <p>Type : <b>${esc(c.type)}</b> | Service : ${esc(c.service || '-')}</p>
       <label>Statut  <select  id="dStatut">${Object.entries(LABELS).filter(([k])  =>  k  !==
'cloture')
       .map(([k,      v])      =>      `<option      value="${k}"      k === c.statut?'selected':" >
{v}</option>`).join('')}</select></label>
       <label>Priorité <select id="dPrio">${['low', 'normal', 'high', 'urgent']
       .map((p)              =>              `<option              p === c.priorite?'selected':" >
{p}</option>`).join('')}</select></label>
       <button id="dSave">Mettre à jour</button>
    <h4>Historique / notes</h4>
             ${c.notes.map((n)    =>    `<div    class="note">${esc(n.texte)}<br><small>${new
Date(n.creeLe).toLocaleString('fr-FR')}         ${n.publique         ?         ''         :
'(interne)'}</small></div>`).join('')}
    <textarea id="dNote" placeholder="Nouvelle note de traitement"></textarea>
    <button id="dAdd">Ajouter la note</button>
    <button id="dDel" class="danger">Supprimer</button>`;
  $('#dSave').onclick = async () => { try { await api('/courriers/' + id, { method: 'PUT',
body:  {  statut:  $('#dStatut').value,  priorite:  $('#dPrio').value  }  });  refresh();
openDetail(id); } catch (e) { alert(e.message); } };
  $('#dAdd').onclick = async () => { try { await api(`/courriers/${id}/notes`, { method:
'POST',  body:  {  texte:  $('#dNote').value  }  });  openDetail(id);  }  catch  (e)  {
alert(e.message); } };
  $('#dDel').onclick = async () => { if (!confirm('Supprimer ce courrier ?')) return; try {
await  api('/courriers/' + id, { method: 'DELETE' }); $('#detail').close(); refresh(); }
catch (e) { alert(e.message); } };
  if (!$('#detail').open) $('#detail').showModal();
}

const refresh = () => { loadStats(); loadList(); };

$('#form').addEventListener('submit', async (ev) => {
  ev.preventDefault();
  const body = Object.fromEntries(new FormData(ev.target));
```

```
  Object.keys(body).forEach((k) => body[k] === '' && delete body[k]);
    try { const c = await api('/courriers', { method: 'POST', body }); say(`Courrier
enregistré : ${c.reference}`); ev.target.reset(); setTimeout(refresh, 1500); }
  catch (e) { say(e.message, false); }
});
$('#btnFiltrer').onclick = () => { page = 1; loadList(); };
$('#prev').onclick = () => { if (page > 1) { page--; loadList(); } };
$('#next').onclick = () => { if (hasNext) { page++; loadList(); } };
$('#liste').addEventListener('click', (e) => { const tr = e.target.closest('tr[data-id]');
if (tr) openDetail(tr.dataset.id); });
$('#fermer').onclick = () => $('#detail').close();
refresh();
```

6. Configuration et lancement

6.1 Variables d'environnement

Variable	Obligatoire	Description
ZENDESK_MODE	Non	Valeur mock pour activer la simulation (aucun compte requis).
ZENDESK_SUBDOMAIN	Mode réel	Sous-domaine du compte (monagence pour monagence.zendesk.com).
ZENDESK_EMAIL	Mode réel	E-mail de l'agent propriétaire du jeton.
ZENDESK_API_TOKEN	Mode réel	Jeton API créé dans Zendesk.
PORT	Non	Port d'écoute, 3000 par défaut.

6.2 Procédure en mode simulation

- Installer Node.js 18 ou plus.
- Décompresser le projet et ouvrir un terminal dans le dossier courriers-zendesk.
- Vérifier que le fichier .env contient ZENDESK_MODE=mock.
- Exécuter npm install, puis npm start.
- Ouvrir http://localhost:3000 dans le navigateur.

6.3 Procédure en mode réel

- Dans Zendesk : Centre d'administration, Apps et intégrations, API, activer l'accès par jeton et créer un jeton.
- Dans .env : supprimer la ligne ZENDESK_MODE et renseigner les trois variables Zendesk.
- Relancer avec npm start. Aucun changement de code n'est nécessaire.

7. Plan de recette

Le tableau suivant liste les cas de test à exécuter. La colonne Résultat est à compléter lors de la recette. Les commandes utilisent curl, un outil en ligne de commande qui envoie des requêtes HTTP ; il est présenté ici avant ses usages.

ID	Cas de test	Action	Résultat attendu	Résultat
T1	Création valide (BF1, BF2)	POST /api/courriers avec type, objet, contenu	201, référence CR-AAAA-XXXXXX	
T2	Type invalide (RG1)	POST avec type = « autre »	400, message de validation	
T3	Objet trop court (RG3)	POST avec objet = « ab »	400	
T4	E-mail invalide (RG6)	POST avec email = « x »	400	
T5	Liste paginée (BF3)	GET /api/courriers	200, tableau et total	
T6	Filtre par type (BF4)	GET /api/courriers?type=entrant	Seuls les entrants	
T7	Recherche mot-clé (BF4)	GET /api/courriers?q=mairie	Courriers contenant « mairie »	
T8	Détail (BF5)	GET /api/courriers/:id	200, notes incluses	
T9	Changement de statut (BF6)	PUT avec statut = en_traitement	200, statut mis à jour	
T10	Changement de priorité (BF7)	PUT avec priorite = urgent	200	
T11	Ajout de note (BF8)	POST /api/courriers/:id/notes	201, note visible au détail	
T12	Suppression (BF9)	DELETE /api/courriers/:id	204, puis 404 au GET	
T13	Statistiques (BF10)	GET /api/stats	Totaux par statut et type	

T14	Identifiant invalide	GET /api/courriers/abc	400	
T15	Route inconnue	GET /api/xyz	404 JSON	
T16	Limitation de débit	Plus de 100 requêtes en 15 min	429	

Exemple de commande pour le test T1 :

```
curl -X POST localhost:3000/api/courriers -H "Content-Type: application/json" -d '{"type":"entrant","objet":"Lettre de la mairie","contenu":"Convocation","priorite":"high"}'
```

8. Sécurité

Menace	Mesure implémentée	Fichier
Fuite du jeton Zendesk	Jeton en variable d'environnement, jamais envoyé au client, .env exclu de Git	zendeskClient.js, .gitignore
Injection XSS	Échappement HTML de toute donnée affichée	app.js (fonction esc)
Entrées malformées	Validation stricte avant tout appel distant	courriers.js
Injection dans la recherche	Filtrage des caractères du mot-clé	courriers.js
Abus et déni de service	Limite de 100 requêtes par 15 min et corps JSON limité à 100 Ko	server.js
En-têtes HTTP faibles	Helmet (CSP et autres)	server.js

Limite connue : l'application n'authentifie pas les agents. Pour la production, il faudra ajouter une authentification (JWT ou SSO) et déployer derrière HTTPS.

9. Difficultés rencontrées et solutions

Difficulté	Solution retenue
Format d'authentification du cours inexact (Listing 1)	Utilisation du format officiel Basic base64(email/token:jeton).
Retard d'indexation de la recherche Zendesk après création	Rafraîchissement différé de 1,5 s dans le frontend.

Dépassement de quota (HTTP 429)	Relance automatique selon l'en-tête Retry-After.
Un ticket closed n'est plus modifiable	Statut « Clôturé » affiché mais non proposé à la modification (RG5).
Compte d'essai Zendesk bloqué par le contrôle anti-fraude	Mode simulation derrière le même adaptateur, activé par ZENDESK_MODE=mock.

10. Métriques du code

Le projet totalise 473 lignes de code réparties sur 9 fichiers. Le graphique et le tableau suivants montrent la répartition par fichier et par couche.

Figure 1 – Nombre de lignes de code par fichier (couleur = couche)

Fichier	Couche	Lignes	Part
src/errors.js	Transverse	17	3.6 %
src/zendeskClient.js	Infrastructure	60	12.7 %
src/zendeskMock.js	Infrastructure	57	12.1 %
src/mapper.js	Service/Mapping	42	8.9 %
src/routes/courriers.js	API	110	23.3 %
src/server.js	Transverse	19	4.0 %
public/index.html	Présentation	62	13.1 %
public/style.css	Présentation	27	5.7 %
public/app.js	Présentation	79	16.7 %
Total		473	100 %

11. Conclusion et évolutions

L'implémentation couvre les dix besoins fonctionnels et applique les patterns du cours : passerelle d'API (le backend), adaptateur (zendeskClient), mapper, relance avec temporisation et sécurité par conception. L'isolation de Zendesk derrière un adaptateur a permis d'ajouter un mode simulation sans toucher aux routes ni au frontend, ce qui illustre la valeur d'une architecture en couches pour les systèmes répartis.

Évolutions possibles :

- authentification des agents (JWT ou SSO) et gestion des rôles ;
- pièces jointes via l'endpoint d'upload de Zendesk ;
- webhooks Zendesk pour des notifications en temps réel ;
- tests automatisés (Jest et Supertest) et conteneurisation Docker ;
- cache des statistiques pour réduire les appels à l'API.

12. Références

- Zendesk Developer Documentation, Zendesk API Reference : developer.zendesk.com/api-reference
- Documentation Node.js : nodejs.org
- Documentation Express : expressjs.com
- Documentation Helmet : helmetjs.github.io
- Cours, Chapitre 4 : Les APIs Business, ENSPD, M. Tchuigoua Elvé.